

dbt Training

 Beginner · Module 01 · 90 min

What is dbt and Why We Use It

From raw SQL chaos to tested, documented, versioned transformations.

The Problem: SQL Pipelines Without Structure

****What happens without dbt**** Stakeholder needs an answer: > ****"I need a monthly report about how many contacts converted to active patients last month?"**** Data team writes pipelines, tables, views and connects it to a BI tool. Code gets stored in personal workspace. Execution is placed in a stored procedure and Snowflake task. Three months later: ****broken****. A column was renamed. No tests. No docs. Two ad-hoc queries produce different numbers.

✗ No single source of truth

Same calculation in 10 different places

✗ No testing

Transformations break silently

✗ No documentation

Tribal knowledge about what columns mean

✗ No dependency management

Nobody knows what breaks when a source changes

What dbt Actually Is

****The one-sentence definition**** dbt is a ****transformation framework**** that lets you write SQL ``SELECT`` statements — and handles materialisation, dependency resolution, testing, and documentation on top of them.

✓ Compiles SQL models into executable statements

✓ Resolves the run order via a DAG

✓ Runs tests against your data

✓ Generates browsable documentation

****dbt does NOT:****

Extract data from HubSpot — that's Lambda

Load data into Snowflake — already done before dbt runs

Schedule itself — that's your orchestrator

Store any data itself

Common confusion: dbt is not a database, not an ETL tool, not a scheduler. It transforms data that is *already in Snowflake*.

dbt Core vs dbt Cloud

|| dbt Core | dbt Fusion | dbt Cloud | |---|---|---|---| | What it is | Open-source CLI tool | Next-gen local runtime (Rust-based, replaces Python engine) | Hosted platform: IDE, scheduler, CI | | **What we use** | **✅ dbt Core** | **⌚ Not yet** | **❌ Not used** | | How we run it | `dbt run`, `dbt test`, `dbt build` | Same CLI commands, much faster | N/A | | How we schedule it | Your orchestrator | Your orchestrator | N/A | | Everything in this training applies to dbt Core — running native in Snowflake or local.

The Data Stack

dbt is the T in ELT: Extract, Load, Transform, Lambda handles E+L



dbt in the Bloomwell data stack

layout: default

background: '#f9f8f5'

dbt Vocabulary

dbt

Data Build Tool

A transformation framework. You write SQL — dbt handles boilerplate code, compilation, run order, testing.

model

A single SQL SELECT statement

Stored as a `.sql` file. dbt compiles it and creates a Snowflake object (table, view, etc.) from it.

materialization

How a model is persisted

The Snowflake object dbt creates from a model. Options: `view`, `table`, `incremental`, `snapshot`.

DAG

Directed Acyclic Graph

The dependency map between models. Determines run order. "Acyclic" means no circular dependencies — model A cannot depend on itself.

lineage

The visual dependency chain

Shows what each model reads from (upstream) and what depends on it (downstream). Generated automatically by dbt docs.

macro

Reusable Jinja-templated SQL

Like a function. Defined once in `macros/`, called from any model. Used to avoid repeating logic across the project.

Project Structure

The basic folder structure applies to all projects.

****Open VS Code — dbt project****

```
analytics/
|
├─ readme.md           ← mandatory. verbose!
├─ .gitignore          ← mandatory, protect .secrets
|
├─ .secrets/           ← tokens, git protected
|
├─ dbt_project.yml      ← project config
├─ profiles.yml         ← in snowflake, local: in ~/.dbt/
|
├─ models/
|   ├─ staging/         ← hubspot__contacts etc.
|   ├─ silver/          ← dim_patient, fct_prescription
|   └─ gold/            ← mrt_monthly_volume
|
├─ snapshots/          ← scd2 models
|
├─ macros/             ← reusable code, like functions
├─ tests/              ← singular tests
└─ target/             ← compiled SQL (git-ignored)
```

dbt_project.yml

Project name, model paths, default materialisation per layer

profiles.yml

Connection to Snowflake — account, role, warehouse. Environments: Prod and Dev.

models/silver/dim_patient.sql

Contains `{{ ref() }}` — we'll cover this in Module 03

target/ folder

Where compiled SQL lives — always git-ignored, never commit it

Exercise: Explore the Project (15 min)

Answer all four questions using only the dbt project — no Googling.

!

Before you start: clone the training repo and create a new Git branch as your personal workspace.

<https://github.com/ThorstenWeberGER/dbt-training-exercise>

Q1

What is the project name defined in `dbt_project.yml` and in which other file you will find this name?

Q2

Find one Silver dimension model (`dim_*`). What table does it reference using `{{ ref() }}` ?

Q3

What is the purpose of the `dbt_project.yml` file and what can you configure here?

Q4

How many models are in `models/gold/` ? List their names.

Q5

Open `dbt_project.yml` . What is the default materialisation for Silver models?

Bonus

Run `dbt ls` in the terminal. How many models are listed?

Debrief: The 3 Core Bullets

01

dbt transforms data that is already in Snowflake — it does not extract or load.

02

Configuration substitutes boilerplate code. dbt handles it, like materialization, tests.

03

Every dbt model is a SELECT statement. dbt wraps it in the right DDL based on materialisation config.

Prep questions for Module 02 are in the module document. You'll be asked them cold at the start of next session.

layout: center

Module 01 Complete

Next: Module 02

Project Setup, Repo Structure & Execution Sequence

Prep: review `dbt_project.yml` before next session